MATH 1203
ODEL DISCRETE MATHEMATICS

Lecture 6: ALGORITHM

Course Lecturer: M. KANYIKA

Applied Studies Department- MUST

March, 2026

Lesson Outcome

By the end of this lesson you should be able to:

- Use algorithms to solve different problems .
- apply the time complexity of the algorithms in solving problems
- solve problems relating to growth functions

INTRODUCTION

- I present to you seventh lecture of this module.
- We will discuss about Algorithm.
- We will construct a model that translates the problem into a mathematical context

Algorithm and their Properties

Definition 1

An algorithm is a finite set of precise instructions for performing a computation or for solving a problem.

Example 2

Describe an algorithm for finding the maximum (largest) value in a finite sequence of integers.

Solution: Method 1, we will simply use the English language to describe the sequence of steps used. We perform the following steps.

Algorithm CONT'D

- Set the temporary maximum equal to the first integer in the sequence.
- Compare the next integer in the sequence to the temporary maximum, and if it is larger than the temporary maximum, set the temporary maximum equal to this integer.
- Repeat the previous step if there are more integers in the sequence.
- Stop when there are no integers left in the sequence. The temporary maximum at this point is the largest integer in the sequence.

In the next slide we will see a practical example that demonstrate method 1

Algorithm example CONT'D

Example 3

Consider the sequence [12, 5, 27, 9, 18, 6, 25] . Describe an algorithm for finding the largest value in a sequence of integers.

Solution:

- Identify the sequence of integer [12, 5, 27, 9, 18, 6, 25]
- Assume the first element 12 is the current largest value.
- Go through each element of the sequence
- Compare the current element with the current largest value (12).
  - $5 > 12$? no
  - $27 > 12$? yes
  - Updated the current largest value as [27,12, 5, 9, 18, 6, 25]
- Move to the next element and do the same

Algorithm example CONT'D

- Move to the next element and do the same
  - $9 > 27$? no
  - $18 > 27$? no
  - $9 > 27$? no
  - $6 > 27$? no
  - $25 > 27$? no
- Once gone through all elements the current largest value in the sequence is 27
- Output the current largest value which is 27

Algorithm : Method 2 of example 1

- Method 2: We can also use computer language using pseudo code description as follows

ALGORITHM 1 Finding the Maximum Element in a Finite Sequence.

procedure max $(a_1, a_2, \ldots, a_n :$ integers $)$

max $:= a_1$

for $i := 2$ to n

  if max $< a_i$ then max $:= a_i$

{max is the largest element }

Properties of Algorithm

- **Input**. An algorithm has input values from a specified set.
- **Output**. It should produce output values from a specified set which is a solution to the problem.
- **Definiteness**. The steps must be defined precisely.
- **Correctness**. The output value should be correct

Properties of Algorithm CONT'D

- **Finiteness**. Produce the desired output after a finite (but perhaps large) number of steps for any input in the set.
- **Effectiveness**. It must be possible to perform each step of an algorithm exactly and in a finite amount of time.
- **Generality**. The procedure should be applicable for all problems of the desired form, not just for a particular set of input values.

Searching Algorithms

- We describe the general searching problem as follows:
  - Locate an element x in a list of distinct elements $a_1, a_2, \ldots, a_n$, or determine that it is not in the list.
  - The solution to this search problem is the location of the term in the list that equals x (that is, i is the solution if $x = a_i$) and is 0 if x is not in the list.

Linear Searching Algorithms

- The linear search algorithm begins by comparing x and a_1.
- When $x = a_1$, the solution is the location of a_1, namely, 1 .
- When $x \neq a_1$, compare x with a_2.
- If $x = a_2$, the solution is the location of a_2,namely, 2.
- When $x \neq a_2$ compare x with a_3.
- Continue this process, comparing x successively with each term of the list until a match is found.
- If the entire list has been searched without locating x, the solution is 0 .

Linear Searching Algorithms CONT'D

Example 4

Given an array integers [5, 9, 3, 7, 2, 1] Find the index target value 7

Solution

- Element 5 is located at position 1
- Compare 5 with target value 7. Since $5 \neq 7$
- Move to second element. Compare 9 and 7 and (index 2).
- Since $9 \neq 7$ Move to third element. Compare 3 and 7 (index 3)
- Since $3 \neq 7$ Move to fourth element. Compare 7 and 7 (index 4)
- Since 7=7 we found the target and return

Binary Searching Algorithms

- Aims at comparing the element to be located to the middle term of the list.
- The list is then split into two smaller sublists of the same size, or where one of these smaller lists has one fewer term than the other.
- The search continues by restricting the search to the appropriate sublist based on the comparison of the element to be located and the middle term.

Binary Search example

Example 5

Search for 19 in the list 1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22 using binary search

Solution:

$$
\begin{array}{llllllllllllllll}
1\ 2\ 3\ 5\ 6\ 7\ 8\ 10 & & & & 12\ 13\ 15\ 16\ 18 & & 19\ 20\ 22 & \&10 & < & 19 \\
 & & & & 12\ 13\ 15\ 16 & & 18\ 1920\ 22 & \&16 & < & 19 \\
 & & & & & & 18\ 19 & 20\ 22 & & \\
 & & & & & & & 18\ 19 & &
\end{array}
$$

Therefore 19 is located as a 14^{th} term in a list.

Sorting

- Sorting is putting the elements into a list in which the elements are in increasing order.
- Sorting the list $7, 2, 1, 4, 5, 9$ produces the list $1, 2, 4, 5, 7, 9$
- Two sorting algorithm include
  - Bubble sort
  - insertion sort

Bubble Sorting

Example 6

Use the Bubble sort to put $3, 2, 4, 1, 5$ into increasing order

increasing order

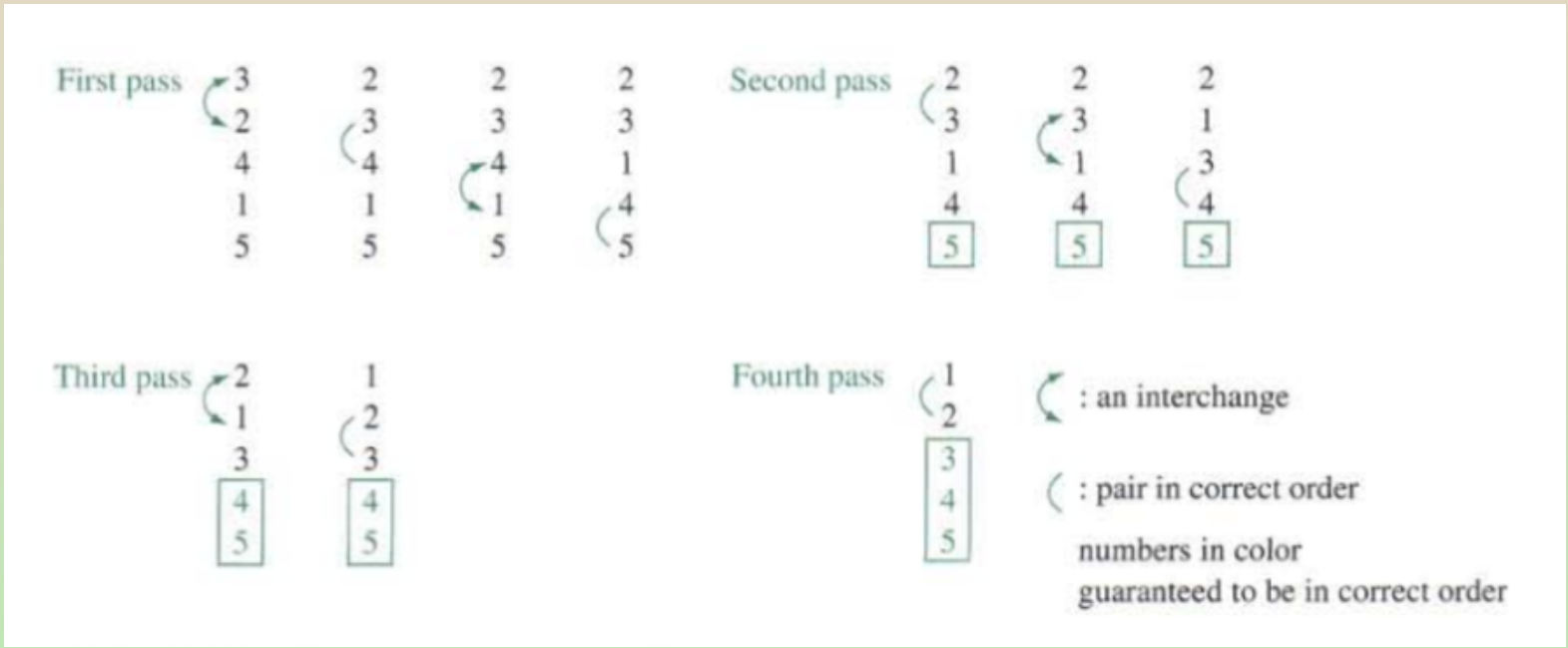
Solution :

- The Steps of a Bubble Sort are as follows:

Bubble Sorting

Solution:

The Steps of a Bubble Sort are as follows:



Insertion Sort

ALGORITHM : The Insertion Sort.

procedure insertion sort $(a_1, a_2, \ldots, a_n$: real numbers with $n \geq 2)$

for $j := 2$ to n

begin

$i := 1$

while $a_j > a_i$

$i := i + 1$

$m := a_j$

for $k := 0$ to $j - i - 1$

$a_{j-k} := a_{j-k-1}$

$a_i := m$

end $\{a_1, a_2, \ldots, a_n$ are sorted $\}$

Insertion Sort Algorithm CONT'D

- Use the Insertion sort to put $3, 2, 4, 1, 5$ into increasing order.

Solution:

$3,\ 2,\ 4,\ 1,\ 5$	The entire list.
$3 > 2$	Compare 2 and 3 from the list.
$2,\ 3,\ 4,\ 1,\ 5$	place 2 in first position of the list.
$4 > 2$ & $4 > 3$	4 placed in third position.
$1 < 2$	compare 1 and 2 from the sorted list
$1,\ 2,\ 3,\ 4,\ 5$	Correct order of the entire list.

Greedy Algorithm

- Greedy algorithm is another type of algorithm that is also used to solve optimization problem.
- Consider the problem of making n cents change with quarters, dimes, nickels, and pennies, and using the least total number of coins.
- We can devise a greedy algorithm for making change for n cents by making a locally optimal choice at each step;

Greedy Algorithm

- Start with an empty list to store the selected coins and set the remaining amount equal to the target amount
- Sort the coin denomonations in descending order
- Iterate through the coin denomination
- The list of selected coins represents the minimum number of coins needed to make up a target amount. ie.
  - 1 penny
  - 5 Nickel
  - 10 dime
  - 25 quarter

Greedy Algorithm

Example 7

Find the minimum number of coins needed to make up a target amount of 67 cents

- Quarter = 67 -25 = 42 cents
  Quarter = 42-25 = 17 cents
  Dime = 17-10= 7cents
  nickel = 7-5=2 cents
  1penny = 2-1= 1 cent
  1penny =1-1= 0 cents
- The minimum number of coins is [25,25,10, 5, 1,1]

Growth functions and time complexity CONT'D

- The growth function is often described using the special Big O notation.
- This notation is used extensively to estimate the number of operations an algorithm uses as its input grows.

Growth functions and time complexity CONT'D

Definition 8

Let f and g be functions from the set of integers or the set of real numbers to the set of real numbers. We say that $f(x)$ is $O(g(x))$ if there are constants C and k such that

$$|f(x)| \leq C|g(x)|$$

whenever $x > k$. [This is read as " $f(x)$ is big-oh of $g(x)$."]

Growth functions and time complexity CONT'D

Example 9

Show that $7x^2$ is $O\left(x^3\right)$.

Solution:

- Note that when $x > 7$, we have $7x^2 < x^3$. (We can obtain this inequality by multiplying both sides of $x > 7$ by x^2.)
- Consequently, we can take $C = 1$ and $k = 7$ as witnesses to establish the relationship $7x^2$ is $O\left(x^3\right)$.
- Alternatively, when $x > 1$, we have $7x^2 < 7x^3$, so that $C = 7$ and $k = 1$ are also witnesses to the relationship $7x^2$ is $O\left(x^3\right)$.

Time complexity

- The time complexity of an algorithm can be expressed in terms of the number of operations used by the algorithm when the input has a particular size instead of actual computer time.
- This is because of the difference in time needed for different computers to perform basic operations.
- The operations used to measure time complexity can be the comparison of integers, the addition of integers, the multiplication of integers, the division of integers, or any other basic operation.

Reading Assignment

- Read through examples of big omega of x
- Examples of time complexity

THANK YOU FOR BEING GOOD AUDIENCE!